

SQL

PALAKURTHI LALITH PRAKASH

2303A52012

26.08.2026

```
create database subqueries_practice;  
use subqueries_practice;
```

-- Table: Departments

```
CREATE TABLE departments (  
dept_id INT PRIMARY KEY,  
dept_name VARCHAR(50),  
location VARCHAR(50)  
);  
INSERT INTO departments VALUES  
(10, 'Engineering', 'Chennai'),  
(20, 'Sales', 'Hyderabad'),  
(30, 'HR', 'Bengaluru'),  
(40, 'Research', 'Pune'),  
(50, 'Support', 'Chennai');
```

-- Table: Employees

```
CREATE TABLE employees (  
emp_id INT PRIMARY KEY,  
emp_name VARCHAR(50),  
dept_id INT,  
salary DECIMAL(10,2),  
manager_id INT,  
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
);  
INSERT INTO employees VALUES  
(101, 'Arun', 10, 50000, 103),  
(102, 'Bala', 20, 45000, 106),  
(103, 'Charan', 10, 80000, NULL),  
(104, 'Divya', 30, 40000, 108),  
(105, 'Ezhil', 10, 65000, 103),  
(106, 'Farah', 20, 75000, NULL),  
(107, 'Gokul', 40, 55000, 109),  
(108, 'Harini', 30, 70000, NULL),  
(109, 'Irfan', 40, 90000, NULL),  
(110, 'Janani', 10, 60000, 103);
```

```
-- Table: Projects
CREATE TABLE projects (
project_id INT PRIMARY KEY,
project_name VARCHAR(50),
dept_id INT,
budget DECIMAL(12,2),
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
INSERT INTO projects VALUES
(1, 'Cloud Migration', 10, 500000),
(2, 'Mobile App', 10, 300000),
(3, 'CRM Upgrade', 20, 250000),
(4, 'Recruitment AI', 30, 200000);
```

```
select * from departments;
```

```
select * from employees;
```

```
select * from projects;
```

```
-- 1
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary BETWEEN 50000 AND 80000
ORDER BY salary DESC, emp_name ASC;
```

```
-- 2
SELECT emp_id, emp_name, dept_id, salary, manager_id
FROM employees
WHERE (
    (dept_id = 10 AND salary > 55000)
    OR
    (dept_id = 20 AND salary >= 70000)
)
AND manager_id IS NOT NULL;
```

```
-- 3
SELECT emp_id, emp_name
FROM employees
```

```
WHERE emp_name LIKE ' _%a%'
AND emp_name NOT LIKE '%n'
ORDER BY emp_name;
```

-- 4

```
SELECT emp_name, salary,
       CASE
         WHEN salary >= 75000 THEN 'High'
         WHEN salary >= 55000 THEN 'Medium'
         ELSE 'Standard'
       END AS salary_band
FROM employees
ORDER BY
  CASE
    WHEN salary >= 75000 THEN 1
    WHEN salary >= 55000 THEN 2
    ELSE 3
  END,
  salary DESC;
```

-- 5

```
SELECT dept_id,
       COUNT(*) AS employee_count,
       MIN(salary) AS min_salary,
       MAX(salary) AS max_salary,
       AVG(salary) AS avg_salary,
       SUM(salary) AS total_salary
FROM employees
GROUP BY dept_id
ORDER BY avg_salary DESC;
```

-- 6

```
SELECT dept_id,
       COUNT(*) AS employee_count,
       AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING COUNT(*) >= 2
AND AVG(salary) > 55000;
```

-- 7

```
SELECT dept_id,  
       COUNT(*) AS project_count,  
       SUM(budget) AS total_budget  
FROM projects  
GROUP BY dept_id  
HAVING SUM(budget) >= 300000;
```

-- 8

```
SELECT e.emp_id,  
       e.emp_name,  
       d.dept_name,  
       d.location,  
       e.salary  
FROM employees e  
JOIN departments d  
ON e.dept_id = d.dept_id  
WHERE e.salary >= 60000  
ORDER BY d.dept_name, e.salary DESC;
```

-- 9

```
SELECT d.dept_id,  
       d.dept_name,  
       COUNT(e.emp_id) AS employee_count  
FROM departments d  
LEFT JOIN employees e  
ON d.dept_id = e.dept_id  
GROUP BY d.dept_id, d.dept_name;
```

-- 10

```
SELECT e.emp_name AS employee_name,  
       e.salary AS employee_salary,  
       m.emp_name AS manager_name,  
       m.salary AS manager_salary  
FROM employees e  
JOIN employees m  
ON e.manager_id = m.emp_id  
ORDER BY m.emp_name, e.salary DESC;
```

-- 11

```
SELECT d.dept_id,  
       d.dept_name,  
       COUNT(p.project_id) AS project_count,  
       COALESCE(SUM(p.budget), 0) AS total_budget  
FROM departments d  
LEFT JOIN projects p  
ON d.dept_id = p.dept_id  
GROUP BY d.dept_id, d.dept_name;
```

-- 12

```
SELECT e.emp_name,  
       d.dept_name,  
       p.project_name,  
       e.salary  
FROM employees e  
JOIN departments d  
ON e.dept_id = d.dept_id  
JOIN projects p  
ON e.dept_id = p.dept_id;
```

-- 13

```
SELECT emp_id, emp_name, salary  
FROM employees  
WHERE salary = (  
    SELECT MAX(salary)  
    FROM employees  
);
```

-- 14

```
SELECT emp_id, emp_name, salary  
FROM employees  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employees  
)  
ORDER BY salary DESC;
```

-- 15

```
SELECT e.emp_id, e.emp_name, e.dept_id, e.salary  
FROM employees e
```

```
WHERE e.salary > (  
    SELECT AVG(e2.salary)  
    FROM employees e2  
    WHERE e2.dept_id = e.dept_id  
);
```

```
-- 16  
SELECT e.emp_id, e.emp_name, e.dept_id, e.salary  
FROM employees e  
WHERE e.salary = (  
    SELECT MAX(e2.salary)  
    FROM employees e2  
    WHERE e2.dept_id = e.dept_id  
);
```

```
-- 17  
SELECT d.dept_id, d.dept_name, d.location  
FROM departments d  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM projects p  
    WHERE p.dept_id = d.dept_id  
);
```

```
-- 18  
SELECT e.emp_id, e.emp_name, e.salary  
FROM employees e  
WHERE e.salary > ALL (  
    SELECT e2.salary  
    FROM employees e2  
    WHERE e2.dept_id = (  
        SELECT d.dept_id  
        FROM departments d  
        WHERE d.dept_name = 'HR'  
    )  
);
```

```
-- 19  
SELECT e.emp_id, e.emp_name, e.dept_id, e.salary  
FROM employees e
```

```

WHERE e.dept_id IN (
    SELECT e2.dept_id
    FROM employees e2
    GROUP BY e2.dept_id
    HAVING AVG(e2.salary) > (
        SELECT AVG(e3.salary)
        FROM employees e3
    )
);

```

```

-- 20
SELECT e.emp_id, e.emp_name, e.salary
FROM employees e
WHERE ABS(
    e.salary - (
        SELECT AVG(salary)
        FROM employees
    )
) = (
    SELECT MIN(
        ABS(
            e2.salary - (
                SELECT AVG(salary)
                FROM employees
            )
        )
    )
    FROM employees e2
);

```